

TEORETICKÁ INFORMATIKA

Problém 3-SAT, problém nezávislé množiny a kliky

David Weber
weber3@spsejecna.cz



Rok 2025/26

Problém 3-SAT

Problém 3-SAT

Problém 3-SAT

- Problém SAT neumíme řešit v polynomiálním čase.

Problém 3-SAT

- Problém SAT neumíme řešit v polynomiálním čase.
- Zkusíme proto původní problém trochu zjednodušit, a to tak, že budeme uvažovat formule v CNF, kde každá klauzule má maximálně 3 literály.

Problém 3-SAT

- Problém SAT neumíme řešit v polynomiálním čase.
- Zkusíme proto původní problém trochu zjednodušit, a to tak, že budeme uvažovat formule v CNF, kde každá klauzule má maximálně 3 literály.

Definice (3-CNF)

Formule φ je v 3-CNF právě tehdy, když je v CNF a každá klauzule obsahuje maximálně 3 literály.

Problém 3-SAT

- Problém SAT neumíme řešit v polynomiálním čase.
- Zkusíme proto původní problém trochu zjednodušit, a to tak, že budeme uvažovat formule v CNF, kde každá klauzule má maximálně 3 literály.

Definice (3-CNF)

Formule φ je v 3-CNF právě tehdy, když je v CNF a každá klauzule obsahuje maximálně 3 literály.

PROBLÉM 3-SAT

Vstup: Formule φ v 3-CNF.

Otázka: Je φ splnitelná?

Problém 3-SAT

Ukážeme, že problém 3-SAT je „stejně těžký“ jako původní SAT.

Ukážeme, že problém 3-SAT je „stejně těžký“ jako původní SAT.

- Jeden problém jsme schopni „převést“ na druhý.

Ukážeme, že problém 3-SAT je „stejně těžký“ jako původní SAT.

- Jeden problém jsme schopni „převést“ na druhý.
- Přesněji k zadané formuli φ v CNF jsme schopni najít formuli ψ v 3-CNF, která je splnitelná právě tehdy, když je i původní formule φ splnitelná.

Ukážeme, že problém 3-SAT je „stejně těžký“ jako původní SAT.

- Jeden problém jsme schopni „převést“ na druhý.
- Přesněji k zadané formuli φ v CNF jsme schopni najít formuli ψ v 3-CNF, která je splnitelná právě tehdy, když je i původní formule φ splnitelná.

Převod mezi formulemi však musíme umět provést
rychle!

Převoditelnost problémů

Polynomiální převoditelnost

- Formálně lze problém chápat jako zobrazení binárního kódování vstupu do množiny $\{0, 1\}$.

Polynomiální převoditelnost

- Formálně lze problém chápat jako zobrazení binárního kódování vstupu do množiny $\{0, 1\}$.
- Existuje celá řada problémů, avšak lze mezi nimi najít zajímavé vztahy.

Polynomiální převoditelnost

- Formálně lze problém chápat jako zobrazení binárního kódování vstupu do množiny $\{0, 1\}$.
- Existuje celá řada problémů, avšak lze mezi nimi najít zajímavé vztahy.
- Resp. některé problémy nemusíme nutně umět řešit přímo $\Rightarrow$ místo toho je převedeme na ty, které řešit umíme!

Polynomiální převoditelnost

- Formálně lze problém chápat jako zobrazení binárního kódování vstupu do množiny $\{0, 1\}$.
- Existuje celá řada problémů, avšak lze mezi nimi najít zajímavé vztahy.
- Resp. některé problémy nemusíme nutně umět řešit přímo $\implies$ místo toho je převedeme na ty, které řešit umíme!

Definice (Polynomiální převoditelnost)

Nechť A, B jsou rozhodovací problémy. Říkáme, že A lze **převést** na B , píšeme $A \rightarrow B$, právě tehdy, když existuje zobrazení $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ takové, že

$$\forall x \in \{0, 1\}^* : A(x) = B(f(x)),$$

a navíc $f(x)$ lze spočítat v polynomiálním čase. Funkci f říkáme **převod** nebo také **redukce**.

Co to znamená?

Co to znamená?

- Problém A je převoditelný na problém B , pokud každý vstup x pro A jsme schopni **rychle sestrojít** odpovídající vstup $f(x)$ pro B .

Co to znamená?

- Problém A je převoditelný na problém B , pokud každý vstup x pro A jsme schopni **rychle sestrojít** odpovídající vstup $f(x)$ pro B .
- Přitom musí platit, že odpověď A na x musí být stejná jako odpověď B na $f(x)$.

Co to znamená?

- Problém A je převoditelný na problém B , pokud každý vstup x pro A jsme schopni **rychle sestavit** odpovídající vstup $f(x)$ pro B .
- Přitom musí platit, že odpověď A na x musí být stejná jako odpověď B na $f(x)$.
- V našem případě: Máme φ v CNF a chceme v polynomiálním čase zkonstruovat formuli ψ v 3-CNF takovou, aby

$$\text{SAT}(\varphi) = 3\text{-SAT}(\psi).$$

Co to znamená?

- Problém A je převoditelný na problém B , pokud každý vstup x pro A jsme schopni **rychle sestrojít** odpovídající vstup $f(x)$ pro B .
- Přitom musí platit, že odpověď A na x musí být stejná jako odpověď B na $f(x)$.
- V našem případě: Máme φ v CNF a chceme v polynomiálním čase zkonstruovat formuli ψ v 3-CNF takovou, aby

$$\text{SAT}(\varphi) = 3\text{-SAT}(\psi).$$

- Tato konstrukce představuje ono zobrazení f , tj. $f(\varphi) = \psi$, nebo-li

$$\text{SAT}(\varphi) = 3\text{-SAT}(f(\varphi)).$$

Poznámky k převoditelnosti

- Nemusí nutně platit, že problémy A a B jsou na sebe **vzájemne** převoditelné.

Poznámky k převoditelnosti

- Nemusí nutně platit, že problémy A a B jsou na sebe **vzájemne** převoditelné.

Tj. $A \rightarrow B$ nutně ještě neznamená, že i $B \rightarrow A$.

Poznámky k převoditelnosti

- Nemusí nutně platit, že problémy A a B jsou na sebe **vzájemne** převoditelné.

Tj. $A \rightarrow B$ nutně ještě neznamená, že i $B \rightarrow A$.

- Např. uvažujme dvojici jednoduchých problémů K, L definovaných následovně:

Poznámky k převoditelnosti

- Nemusí nutně platit, že problémy A a B jsou na sebe **vzájemne** převoditelné.

Tj. $A \rightarrow B$ nutně ještě neznamená, že i $B \rightarrow A$.

- Např. uvažujme dvojici jednoduchých problémů K, L definovaných následovně:

$K(x) = 0$ Na každý vstup odpověz nulou,

Poznámky k převoditelnosti

- Nemusí nutně platit, že problémy A a B jsou na sebe **vzájemne** převoditelné.

Tj. $A \rightarrow B$ nutně ještě neznamená, že i $B \rightarrow A$.

- Např. uvažujme dvojici jednoduchých problémů K, L definovaných následovně:

$K(x) = 0$	Na každý vstup odpověz nulou,
$L(x) = 1$	Na každý vstup odpověz jedničkou.

Poznámky k převoditelnosti

- Nemusí nutně platit, že problémy A a B jsou na sebe **vzájemne** převoditelné.

Tj. $A \rightarrow B$ nutně ještě neznamena, že i $B \rightarrow A$.

- Např. uvažujme dvojici jednoduchých problémů K, L definovaných následovně:

$K(x) = 0$	Na každý vstup odpověz nulou,
$L(x) = 1$	Na každý vstup odpověz jedničkou.

Takovou dvojici problémů na sebe nepřevedeme **ani**
jedním směrem.

Převod SAT $\rightarrow$ HALT

Převod SAT $\rightarrow$ HALT

Převod SAT na HALT lze provést jednoduše: Sestrojíme program P se vstupem φ , který

Převod SAT $\rightarrow$ HALT

Převod SAT na HALT lze provést jednoduše: Sestrojíme program P se vstupem φ , který

- 1 postupně zkouší všechna ohodnocení proměnných formule φ ,

Převod SAT $\rightarrow$ HALT

Převod SAT na HALT lze provést jednoduše: Sestrojíme program P se vstupem φ , který

- 1 postupně zkouší všechna ohodnocení proměnných formule φ ,
- 2 pokud najde splňující ohodnocení, **zastaví se**,

Převod SAT $\rightarrow$ HALT

Převod SAT na HALT lze provést jednoduše: Sestrojíme program P se vstupem φ , který

- 1 postupně zkouší všechna ohodnocení proměnných formule φ ,
- 2 pokud najde splňující ohodnocení, **zastaví se**,
- 3 jinak vstoupí do nekonečného cyklu.

Převod SAT $\rightarrow$ HALT

Převod SAT na HALT lze provést jednoduše: Sestrojíme program P se vstupem φ , který

- 1 postupně zkouší všechna ohodnocení proměnných formule φ ,
- 2 pokud najde splňující ohodnocení, **zastaví se**,
- 3 jinak vstoupí do nekonečného cyklu.

Text programu jsme schopni sestrojit **v konstantním čase**.

Převod SAT $\rightarrow$ HALT

Převod SAT na HALT lze provést jednoduše: Sestrojíme program P se vstupem φ , který

- 1 postupně zkouší všechna ohodnocení proměnných formule φ ,
- 2 pokud najde splňující ohodnocení, **zastaví se**,
- 3 jinak vstoupí do nekonečného cyklu.

Text programu jsme schopni sestrojit **v konstantním čase**.

Tedy skutečně platí, že

φ je splnitelná $\implies$ program P se nad φ zastaví,

nebo-li $\text{SAT}(\varphi) = 1 \implies \text{HALT}(P, \varphi) = 1$.

Převod $\text{HALT} \rightarrow \text{SAT}$ není možný

Převod $\text{HALT} \rightarrow \text{SAT}$ není možný

- Už jsme si zdůvodnili, že ne pro všechny páry (P, x) lze rozhodnout, zda se program P nad x zastaví.

Převod $\text{HALT} \rightarrow \text{SAT}$ není možný

- Už jsme si zdůvodnili, že ne pro všechny páry (P, x) lze rozhodnout, zda se program P nad x zastaví.
- Konkrétně myšlenka byla založena na konstrukci programu D , který invertoval výstup HALT .

Převod $\text{HALT} \rightarrow \text{SAT}$ není možný

- Už jsme si zdůvodnili, že ne pro všechny páry (P, x) lze rozhodnout, zda se program P nad x zastaví.
- Konkrétně myšlenka byla založena na konstrukci programu D , který invertoval výstup HALT .
- Pro spor předpokládejme, že umíme přesvést **problém zastavení** na pro **problém splnitelnosti formule v CNF**.

Převod $\text{HALT} \rightarrow \text{SAT}$ není možný

- Už jsme si zdůvodnili, že ne pro všechny páry (P, x) lze rozhodnout, zda se program P nad x zastaví.
- Konkrétně myšlenka byla založena na konstrukci programu D , který invertoval výstup HALT .
- Pro spor předpokládejme, že umíme přesvést **problém zastavení** na pro **problém splnitelnosti formule v CNF**.
 - 1 Instanci (P, x) převedeme polynomiálně na formuli $\varphi = f(P, x)$.

Převod $\text{HALT} \rightarrow \text{SAT}$ není možný

- Už jsme si zdůvodnili, že ne pro všechny páry (P, x) lze rozhodnout, zda se program P nad x zastaví.
- Konkrétně myšlenka byla založena na konstrukci programu D , který invertoval výstup HALT .
- Pro spor předpokládejme, že umíme přesvést **problém zastavení** na pro **problém splnitelnosti formule v CNF**.
 - 1 Instanci (P, x) převedeme polynomiálně na formuli $\varphi = f(P, x)$.
 - 2 Následně rozhodneme, zda $\text{SAT}(f(P, x)) = 1$, tj. zdali je formule $f(P, x)$ splnitelná.

Převod $\text{HALT} \rightarrow \text{SAT}$ není možný

- Už jsme si zdůvodnili, že ne pro všechny páry (P, x) lze rozhodnout, zda se program P nad x zastaví.
- Konkrétně myšlenka byla založena na konstrukci programu D , který invertoval výstup HALT .
- Pro spor předpokládejme, že umíme přesvést **problém zastavení** na pro **problém splnitelnosti formule v CNF**.
 - 1 Instanci (P, x) převedeme polynomiálně na formuli $\varphi = f(P, x)$.
 - 2 Následně rozhodneme, zda $\text{SAT}(f(P, x)) = 1$, tj. zdali je formule $f(P, x)$ splnitelná.

Jenže pro $P = D$ nelze problém zastavení rozhodnout, zatímco v tomto případě bychom jednoduše mohli sestavit formuli $\psi = f(D, D)$ a rozhodnout, zdali $\text{SAT}(\psi) = 1$. **To je spor!**

Převod mezi SAT a 3-SAT

Převod 3-SAT $\rightarrow$ SAT

Převod 3-SAT $\rightarrow$ SAT

- Na začátku máme formuli φ v 3-CNF a chceme k ní sestrojit formuli ψ v CNF, která je splnitelná právě tehdy, když je splnitelná i φ .

Převod 3-SAT $\rightarrow$ SAT

- Na začátku máme formuli φ v 3-CNF a chceme k ní sestrojit formuli ψ v CNF, která je splnitelná právě tehdy, když je splnitelná i φ .

Každá formule ve 3-CNF je zároveň i v CNF $\Rightarrow$
ponecháme původní formuli φ .

Převod 3-SAT $\rightarrow$ SAT

- Na začátku máme formuli φ v 3-CNF a chceme k ní sestrojit formuli ψ v CNF, která je splnitelná právě tehdy, když je splnitelná i φ .

Každá formule ve 3-CNF je zároveň i v CNF $\Rightarrow$
ponecháme původní formuli φ .

- Za převodní zobrazení f stačí zvolit identitu, tj. $f(x) = x$.

Převod $SAT \rightarrow 3-SAT$

Převod SAT $\rightarrow$ 3-SAT

- Zde je již zjevné, že ne každá formule φ v CNF je automaticky i v 3-CNF.

Převod SAT $\rightarrow$ 3-SAT

- Zde je již zjevné, že ne každá formule φ v CNF je automaticky i v 3-CNF.
- U vstupní formule proto musíme rozdělit „dlouhé“ klauzule na menší.

Převod SAT $\rightarrow$ 3-SAT

- Zde je již zjevné, že ne každá formule φ v CNF je automaticky i v 3-CNF.
- U vstupní formule proto musíme rozdělit „dlouhé“ klauzule na menší.
- Na vstupu tedy máme formuli φ v CNF. Mohou nastat 2 scénáře:

Převod SAT $\rightarrow$ 3-SAT

- Zde je již zjevné, že ne každá formule φ v CNF je automaticky i v 3-CNF.
- U vstupní formule proto musíme rozdělit „dlouhé“ klauzule na menší.
- Na vstupu tedy máme formuli φ v CNF. Mohou nastat 2 scénáře:
 - 1 Formule φ neobsahuje klauzuli s více než 3 literály. Pak jsme hotovi. ✓

Převod SAT $\rightarrow$ 3-SAT

- Zde je již zjevné, že ne každá formule φ v CNF je automaticky i v 3-CNF.
- U vstupní formule proto musíme rozdělit „dlouhé“ klauzule na menší.
- Na vstupu tedy máme formuli φ v CNF. Mohou nastat 2 scénáře:
 - 1 Formule φ neobsahuje klauzuli s více než 3 literály. Pak jsme hotovi. ✓
 - 2 Jinak vybereme nějakou klauzuli C , která obsahuje $k > 3$ literálů a rozdělíme ji dvě části α, β :

$$C = (\underbrace{\ell_1 \vee \ell_2}_{\alpha} \vee \overbrace{\ell_3 \vee \dots \vee \ell_k}^{\beta}).$$

Tedy $\alpha = \ell_1 \vee \ell_2$ a $\beta = \ell_3 \vee \dots \vee \ell_k$.

Nové klauzule s α a β

Nové klauzule s α a β

- Přidáme novou proměnnou x a pro původní klauzuli zapíšeme pomocí dvojice nových klauzulí následovně:

$$(\ell_1 \vee \ell_2 \vee \dots \vee \ell_k) \equiv (\alpha \vee x) \wedge (\beta \vee \neg x).$$

Nové klauzule s α a β

- Přidáme novou proměnnou x a pro původní klauzuli zapíšeme pomocí dvojice nových klauzulí následovně:

$$(\ell_1 \vee \ell_2 \vee \dots \vee \ell_k) \equiv (\alpha \vee x) \wedge (\beta \vee \neg x).$$

- Všimněme si, že

Nové klauzule s α a β

- Přidáme novou proměnnou x a pro původní klauzuli zapíšeme pomocí dvojice nových klauzulí následovně:

$$(\ell_1 \vee \ell_2 \vee \dots \vee \ell_k) \equiv (\alpha \vee x) \wedge (\beta \vee \neg x).$$

- Všimněme si, že
 - Klauzule $(\alpha \vee x) = (\ell_1 \vee \ell_2 \vee x)$ má 3 literály.

Nové klauzule s α a β

- Přidáme novou proměnnou x a pro původní klauzuli zapíšeme pomocí dvojice nových klauzulí následovně:

$$(\ell_1 \vee \ell_2 \vee \dots \vee \ell_k) \equiv (\alpha \vee x) \wedge (\beta \vee \neg x).$$

- Všimněme si, že
 - Klauzule $(\alpha \vee x) = (\ell_1 \vee \ell_2 \vee x)$ má 3 literály.
 - Klauzule $(\beta \vee \neg x) = (\ell_3 \vee \dots \vee \ell_k \vee \neg x)$ má $k - 1$ literálů.

Nové klauzule s α a β

- Přidáme novou proměnnou x a pro původní klauzuli zapíšeme pomocí dvojice nových klauzulí následovně:

$$(\ell_1 \vee \ell_2 \vee \dots \vee \ell_k) \equiv (\alpha \vee x) \wedge (\beta \vee \neg x).$$

- Všimněme si, že
 - Klauzule $(\alpha \vee x) = (\ell_1 \vee \ell_2 \vee x)$ má 3 literály.
 - Klauzule $(\beta \vee \neg x) = (\ell_3 \vee \dots \vee \ell_k \vee \neg x)$ má $k - 1$ literálů.

Původní klauzuli jsme rozdělili na **dvojici kratších klauzulí**.

Nové klauzule s α a β

- Přidáme novou proměnnou x a pro původní klauzuli zapíšeme pomocí dvojice nových klauzulí následovně:

$$(\ell_1 \vee \ell_2 \vee \dots \vee \ell_k) \equiv (\alpha \vee x) \wedge (\beta \vee \neg x).$$

- Všimněme si, že
 - Klauzule $(\alpha \vee x) = (\ell_1 \vee \ell_2 \vee x)$ má 3 literály.
 - Klauzule $(\beta \vee \neg x) = (\ell_3 \vee \dots \vee \ell_k \vee \neg x)$ má $k - 1$ literálů.

Původní klauzuli jsme rozdělili na **dvojici kratších klauzulí**.

- Pokud je klauzule $(\beta \vee \neg x)$ stále dlouhá, tak postup opakujeme.

Hodnota proměnné x

- Nyní zbývá dořešit, jak nastavit hodnotu proměnné x .
 - 1 Pokud $\alpha = 1$, pak je klauzule $(\alpha \vee x)$ splněna $\implies$ nastavením $x = 0$ splníme i druhou klauzuli $(\beta \vee \neg x)$.
 - 2 Pokud $\alpha = 0$, pak nutně $\beta = 1$. Jinak by původní klauzule

$$(\ell_1 \vee \dots \vee \ell_k) = (\alpha \vee \beta)$$

nebyla splněna. $\implies$ Nastavíme $x = 1$, abychom splnili klauzuli $(\alpha \vee x)$.

Závěrem k převodu $SAT \rightarrow 3-SAT$

Závěrem k převodu $SAT \rightarrow 3-SAT$

- Opakovanou aplikací tohoto postupu dostaneme formuli ψ , která je v 3-CNF a je splnitelná právě tehdy, když i φ je splnitelná.

Závěrem k převodu SAT $\rightarrow$ 3-SAT

- Opakovanou aplikací tohoto postupu dostaneme formuli ψ , která je v 3-CNF a je splnitelná právě tehdy, když i φ je splnitelná.
- Rozdělení klauzule

$$C = (\ell_1 \vee \ell_2 \vee \dots \vee \ell_k)$$

délky k provedeme v $k - 3$ krocích. Tím postupně dostaneme

$$C \equiv (\ell_1 \vee \ell_2 \vee x_1) \wedge (\neg x_1 \vee \ell_3 \vee x_2) \wedge \dots \wedge (\neg x_{k-4} \vee \ell_{k-2} \vee x_{k-3}) \\ \wedge (\neg x_{k-3} \vee \ell_{k-1} \vee \ell_k),$$

kde $x_1, x_2, \dots, x_{k-3}$ jsou nové proměnné.

Závěrem k převodu SAT $\rightarrow$ 3-SAT

- Opakovanou aplikací tohoto postupu dostaneme formuli ψ , která je v 3-CNF a je splnitelná právě tehdy, když i φ je splnitelná.
- Rozdělení klauzule

$$C = (\ell_1 \vee \ell_2 \vee \dots \vee \ell_k)$$

délky k provedeme v $k - 3$ krocích. Tím postupně dostaneme

$$C \equiv (\ell_1 \vee \ell_2 \vee x_1) \wedge (\neg x_1 \vee \ell_3 \vee x_2) \wedge \dots \wedge (\neg x_{k-4} \vee \ell_{k-2} \vee x_{k-3}) \\ \wedge (\neg x_{k-3} \vee \ell_{k-1} \vee \ell_k),$$

kde $x_1, x_2, \dots, x_{k-3}$ jsou nové proměnné.

- Tento rozklad potvrzuje lineárně dlouho vzhledem k délce klauzule, tj. $O(k)$.

Závěrem k převodu SAT $\rightarrow$ 3-SAT

- Opakovanou aplikací tohoto postupu dostaneme formuli ψ , která je v 3-CNF a je splnitelná právě tehdy, když i φ je splnitelná.
- Rozdělení klauzule

$$C = (\ell_1 \vee \ell_2 \vee \dots \vee \ell_k)$$

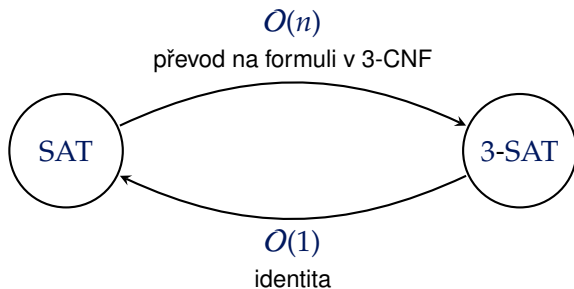
délky k provedeme v $k - 3$ krocích. Tím postupně dostaneme

$$C \equiv (\ell_1 \vee \ell_2 \vee x_1) \wedge (\neg x_1 \vee \ell_3 \vee x_2) \wedge \dots \wedge (\neg x_{k-4} \vee \ell_{k-2} \vee x_{k-3}) \\ \wedge (\neg x_{k-3} \vee \ell_{k-1} \vee \ell_k),$$

kde $x_1, x_2, \dots, x_{k-3}$ jsou nové proměnné.

- Tento rozklad potrvá lineárně dlouho vzhledem k délce klauzule, tj. $O(k)$.
- Celkově převod všech dlouhých klauzulí tedy potrvá $O(n)$, kde n je počet literálů.

SAT a 3-SAT jsou vzájemně převoditelné



Převod SAT $\rightarrow$ 3-SAT

Příklad

Mějme formuli φ v CNF s jednou klauzulí:

$$\varphi = a \vee b \vee \neg c \vee \neg b \vee d \vee \neg d.$$

- Rozdělíme φ na dvě klauzule pomocí nové proměnné x :

$$(a \vee b \vee x) \wedge (\neg c \vee \neg b \vee d \vee \neg d \vee \neg x).$$

- Pro druhou klauzuli nyní opakujeme stejný postup: Zavedeme si novou proměnnou y a rozdělíme ji na dvě klauzule, tj.

$$(a \vee b \vee x) \wedge (\neg c \vee \neg b \vee y) \wedge (d \vee \neg d \vee \neg x \vee \neg y).$$

- A nyní ještě poslední klauzuli rozdělíme pomocí nové proměnné z :

$$(a \vee b \vee x) \wedge (\neg c \vee \neg b \vee y) \wedge (d \vee \neg d \vee z) \wedge (\neg x \vee \neg y \vee \neg z).$$

Převod $SAT \rightarrow 3\text{-}SAT$

Příklad

Převod SAT $\rightarrow$ 3-SAT

Příklad

Výsledná formule v 3-CNF je tedy

$$\psi = (a \vee b \vee x) \wedge (\neg c \vee \neg b \vee y) \wedge (d \vee \neg d \vee z) \wedge (\neg x \vee \neg y \vee \neg z).$$

Převod SAT $\rightarrow$ 3-SAT

Příklad

Výsledná formule v 3-CNF je tedy

$$\psi = (a \vee b \vee x) \wedge (\neg c \vee \neg b \vee y) \wedge (d \vee \neg d \vee z) \wedge (\neg x \vee \neg y \vee \neg z).$$

Zkusíme ohodnocení proměnných $a = 0, b = 1, c = 1, d = 1$. Jak musíme nastavit x, y, z ?

Převod SAT $\rightarrow$ 3-SAT

Příklad

Výsledná formule v 3-CNF je tedy

$$\psi = (a \vee b \vee x) \wedge (\neg c \vee \neg b \vee y) \wedge (d \vee \neg d \vee z) \wedge (\neg x \vee \neg y \vee \neg z).$$

Zkusíme ohodnocení proměnných $a = 0, b = 1, c = 1, d = 1$. Jak musíme nastavit x, y, z ?

- První klauzule: $a \vee b = 0 \vee 1 = 1$, tedy nastavíme $x = 0$.

Převod SAT $\rightarrow$ 3-SAT

Příklad

Výsledná formule v 3-CNF je tedy

$$\psi = (a \vee b \vee x) \wedge (\neg c \vee \neg b \vee y) \wedge (d \vee \neg d \vee z) \wedge (\neg x \vee \neg y \vee \neg z).$$

Zkusíme ohodnocení proměnných $a = 0, b = 1, c = 1, d = 1$. Jak musíme nastavit x, y, z ?

- První klauzule: $a \vee b = 0 \vee 1 = 1$, tedy nastavíme $x = 0$.
- Druhá klauzule: $\neg c \vee \neg b = \neg 1 \vee \neg 1 = 0$, nastavíme $y = 1$.

Převod SAT $\rightarrow$ 3-SAT

Příklad

Výsledná formule v 3-CNF je tedy

$$\psi = (a \vee b \vee x) \wedge (\neg c \vee \neg b \vee y) \wedge (d \vee \neg d \vee z) \wedge (\neg x \vee \neg y \vee \neg z).$$

Zkusíme ohodnocení proměnných $a = 0, b = 1, c = 1, d = 1$. Jak musíme nastavit x, y, z ?

- První klauzule: $a \vee b = 0 \vee 1 = 1$, tedy nastavíme $x = 0$.
- Druhá klauzule: $\neg c \vee \neg b = \neg 1 \vee \neg 1 = 0$, nastavíme $y = 1$.
- Třetí klauzule: $d \vee \neg d = 1 \vee \neg 1 = 1$, nastavíme $z = 0$.

Převod SAT $\rightarrow$ 3-SAT

Příklad

Výsledná formule v 3-CNF je tedy

$$\psi = (a \vee b \vee x) \wedge (\neg c \vee \neg b \vee y) \wedge (d \vee \neg d \vee z) \wedge (\neg x \vee \neg y \vee \neg z).$$

Zkusíme ohodnocení proměnných $a = 0, b = 1, c = 1, d = 1$. Jak musíme nastavit x, y, z ?

- První klauzule: $a \vee b = 0 \vee 1 = 1$, tedy nastavíme $x = 0$.
- Druhá klauzule: $\neg c \vee \neg b = \neg 1 \vee \neg 1 = 0$, nastavíme $y = 1$.
- Třetí klauzule: $d \vee \neg d = 1 \vee \neg 1 = 1$, nastavíme $z = 0$.
- Čtvrtá klauzule: $(\neg x \vee \neg y \vee \neg z) = \neg 0 \vee \neg 1 \vee \neg 0 = 1$.

Převod SAT $\rightarrow$ 3-SAT

Příklad

Výsledná formule v 3-CNF je tedy

$$\psi = (a \vee b \vee x) \wedge (\neg c \vee \neg b \vee y) \wedge (d \vee \neg d \vee z) \wedge (\neg x \vee \neg y \vee \neg z).$$

Zkusíme ohodnocení proměnných $a = 0, b = 1, c = 1, d = 1$. Jak musíme nastavit x, y, z ?

- První klauzule: $a \vee b = 0 \vee 1 = 1$, tedy nastavíme $x = 0$.
- Druhá klauzule: $\neg c \vee \neg b = \neg 1 \vee \neg 1 = 0$, nastavíme $y = 1$.
- Třetí klauzule: $d \vee \neg d = 1 \vee \neg 1 = 1$, nastavíme $z = 0$.
- Čtvrtá klauzule: $(\neg x \vee \neg y \vee \neg z) = \neg 0 \vee \neg 1 \vee \neg 0 = 1$.

Platí, že $\varphi(\dots) = \psi(\dots) = 1$.

Obtížnost problémů

Převoditelnost a „obtížnost“ problému

- $A \rightarrow B$ v podstatě říká, že problém B alespoň tak těžký jako problém A .

- $A \rightarrow B$ v podstatě říká, že problém B alespoň tak těžký jako problém A .

Pokud umíme vyřešit B , pak vyřešit A je nanejvýše „polynomiálně obtížnější“.

Převoditelnost a „obtížnost“ problému

- $A \rightarrow B$ v podstatě říká, že problém B alespoň tak těžký jako problém A .

Pokud umíme vyřešit B , pak vyřešit A je nanejvýše „polynomiálně obtížnější“.

- Zároveň to znamená, že pomocí problému B můžeme vyřešit A s maximálně polynomiálním zpomalením (kvůli převodu vstupu).

Převoditelnost a „obtížnost“ problému

- $A \rightarrow B$ v podstatě říká, že problém B alespoň tak těžký jako problém A .

Pokud umíme vyřešit B , pak vyřešit A je nanejvýše „polynomiálně obtížnější“.

- Zároveň to znamená, že pomocí problému B můžeme vyřešit A s maximálně polynomiálním zpomalením (kvůli převodu vstupu).
- Mnemotechnická pomůcka: *Obtížnost teče ve směru šipky.*

Umíme-li B rychle, umíme i A rychle

Umíme-li B rychle, umíme i A rychle

Tvrzení

Pokud $A \rightarrow B$ a B lze řešit v polynomiálním čase, pak i A lze řešit v polynomiálním čase.

Umíme-li B rychle, umíme i A rychle

Tvrzení

Pokud $A \rightarrow B$ a B lze řešit v polynomiálním čase, pak i A lze řešit v polynomiálním čase.

- Uvažujme algoritmus řešící problém B v čase $O(b^k)$, kde b je délka vstupu tohoto problému a $k \in \mathbb{R}^+$.

Umíme-li B rychle, umíme i A rychle

Tvrzení

Pokud $A \rightarrow B$ a B lze řešit v polynomiálním čase, pak i A lze řešit v polynomiálním čase.

- Uvažujme algoritmus řešící problém B v čase $O(b^k)$, kde b je délka vstupu tohoto problému a $k \in \mathbb{R}^+$.
- Dále mějme zobrazení f převádějící A na B v čase $O(a^\ell)$ pro vstup délky a a konstantu $\ell \in \mathbb{R}^+$.

Umíme-li B rychle, umíme i A rychle

Tvrzení

Pokud $A \rightarrow B$ a B lze řešit v polynomiálním čase, pak i A lze řešit v polynomiálním čase.

- Uvažujme algoritmus řešící problém B v čase $O(b^k)$, kde b je délka vstupu tohoto problému a $k \in \mathbb{R}^+$.
- Dále mějme zobrazení f převádějící A na B v čase $O(a^\ell)$ pro vstup délky a a konstantu $\ell \in \mathbb{R}^+$.
- Chceme-li spočítat $A(x)$ pro vstup x délky a , spočítáme nejprve $f(x)$ v čase $O(a^\ell)$

Umíme-li B rychle, umíme i A rychle

Tvrzení

Pokud $A \rightarrow B$ a B lze řešit v polynomiálním čase, pak i A lze řešit v polynomiálním čase.

- Uvažujme algoritmus řešící problém B v čase $O(b^k)$, kde b je délka vstupu tohoto problému a $k \in \mathbb{R}^+$.
- Dále mějme zobrazení f převádějící A na B v čase $O(a^\ell)$ pro vstup délky a a konstantu $\ell \in \mathbb{R}^+$.
- Chceme-li spočítat $A(x)$ pro vstup x délky a , spočítáme nejprve $f(x)$ v čase $O(a^\ell)$
 $\implies$ Tím dostaneme výstup délky $O(a^\ell)$ (delší bychom ani nestihli vypsát).

Umíme-li B rychle, umíme i A rychle

Tvrzení

Pokud $A \rightarrow B$ a B lze řešit v polynomiálním čase, pak i A lze řešit v polynomiálním čase.

- Uvažujme algoritmus řešící problém B v čase $O(b^k)$, kde b je délka vstupu tohoto problému a $k \in \mathbb{R}^+$.
- Dále mějme zobrazení f převádějící A na B v čase $O(a^\ell)$ pro vstup délky a a konstantu $\ell \in \mathbb{R}^+$.
- Chceme-li spočítat $A(x)$ pro vstup x délky a , spočítáme nejprve $f(x)$ v čase $O(a^\ell)$
 $\implies$ Tím dostaneme výstup délky $O(a^\ell)$ (delší bychom ani nestihli vypsát).
- Tento výstup $f(x)$ předáme na vstup algoritmu řešící problém B , který nad ním stráví $O((a^\ell)^k) = O(a^{k\ell})$.

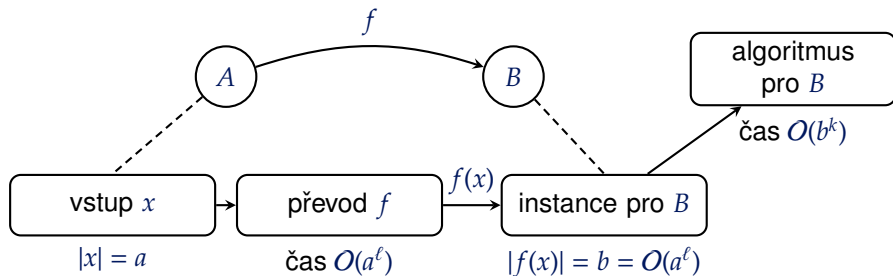
Umíme-li B rychle, umíme i A rychle

Tvrzení

Pokud $A \rightarrow B$ a B lze řešit v polynomiálním čase, pak i A lze řešit v polynomiálním čase.

- Uvažujme algoritmus řešící problém B v čase $O(b^k)$, kde b je délka vstupu tohoto problému a $k \in \mathbb{R}^+$.
- Dále mějme zobrazení f převádějící A na B v čase $O(a^\ell)$ pro vstup délky a a konstantu $\ell \in \mathbb{R}^+$.
- Chceme-li spočítat $A(x)$ pro vstup x délky a , spočítáme nejprve $f(x)$ v čase $O(a^\ell)$
 $\implies$ Tím dostaneme výstup délky $O(a^\ell)$ (delší bychom ani nestihli vypsát).
- Tento výstup $f(x)$ předáme na vstup algoritmu řešící problém B , který nad ním stráví $O((a^\ell)^k) = O(a^{k\ell})$.
- Celkově strávíme výpočtem $O(a^\ell + a^{k\ell})$, což je polynom v délce původního vstupu x .

Znázornění tvrzení



$$O(b^k) = O((a^\ell)^k) = O(a^{k\ell})$$

$$\text{Celkem: } O(a^\ell + a^{k\ell})$$

Co z toho plyne?

Co z toho plyne?

Speciálně to znamená, že problém **3-SAT** a **SAT** jsou
stejně těžké problémy.

Co z toho plyne?

Speciálně to znamená, že problém **3-SAT** a **SAT** jsou **stejně těžké problémy**.

Zkrácení délky klauzulí pro nás bude mít jednu zásadní výhodu u dalších problémů.

Problém nezávislé množiny

Nezávislá množina

Nezávislá množina

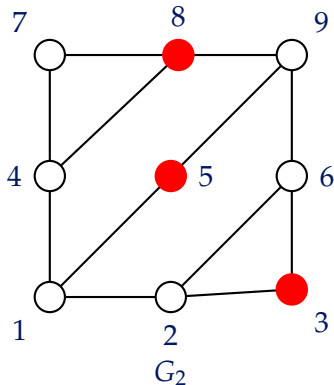
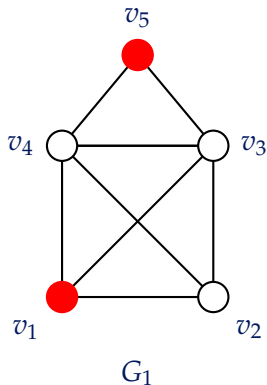
Definice (Nezávislá množina)

Mějme libovolný graf $G = (V, E)$. Množina vrcholů $M \subseteq V$ je **nezávislá**, pokud žádné dva vrcholy $u, v \in M$ nejsou spojeny hranou.

Nezávislá množina

Definice (Nezávislá množina)

Mějme libovolný graf $G = (V, E)$. Množina vrcholů $M \subseteq V$ je **nezávislá**, pokud žádné dva vrcholy $u, v \in M$ nejsou spojeny hranou.



Existence nezávislé množiny

Existence nezávislé množiny

Na samotnou existenci nezávislé množiny nemá moc smysl se ptát
 $\Rightarrow$ stačí zkrátka vzít **jednovrcholovou** nebo **prázdnou** množinu.

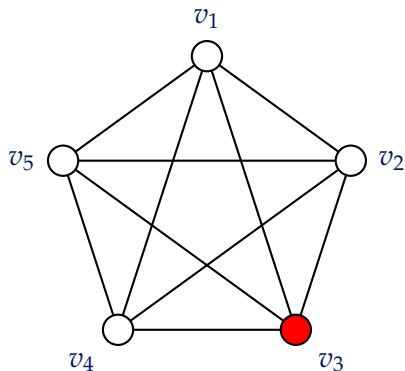
Existence nezávislé množiny

Na samotnou existenci nezávislé množiny nemá moc smysl se ptát
⇒ stačí zkrátka vzít **jednovrcholovou** nebo **prázdnou** množinu.

Zajímavější je však otázka, zdali existuje nezávislá množina **určité velikosti**.

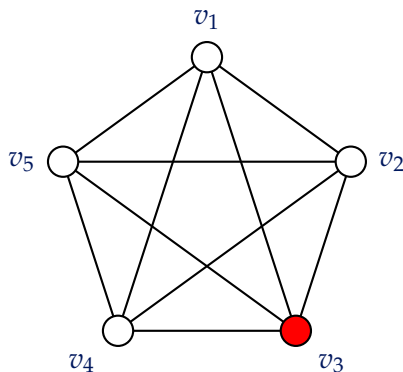
Nezávislá množina

Příklad: Úplný graf



Nezávislá množina

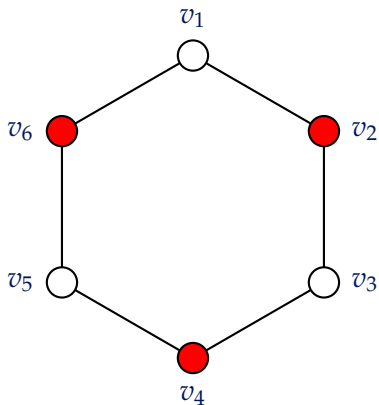
Příklad: Úplný graf



Největší nezávislá v úplném grafu obsahuje pouze jeden vrchol.

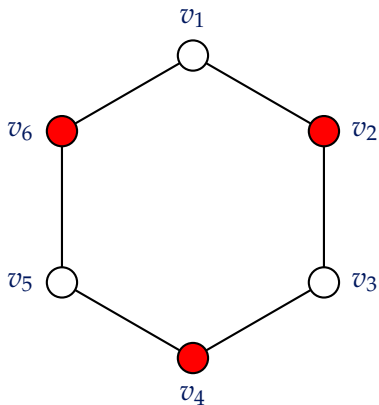
Nezávislá množina

Příklad: Cyklus



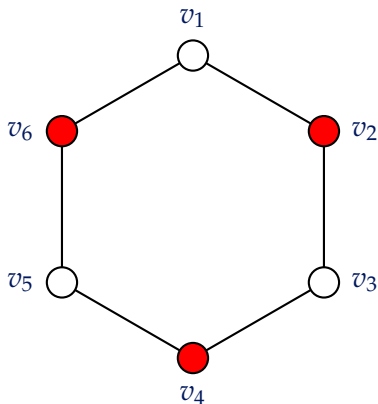
Nezávislá množina

Příklad: Cyklus



Nezávislá množina

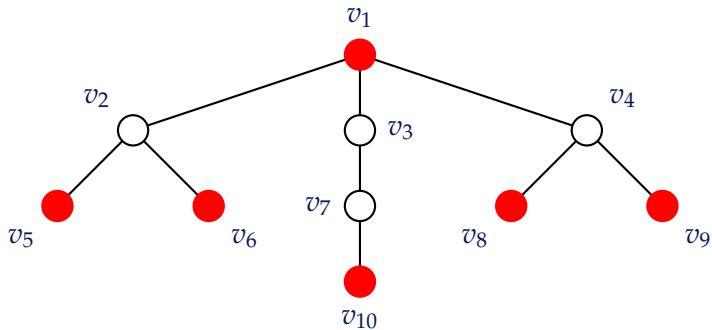
Příklad: Cyklus



Největší nezávislá množina v cyklu C_n má velikost $\lfloor n/2 \rfloor$.

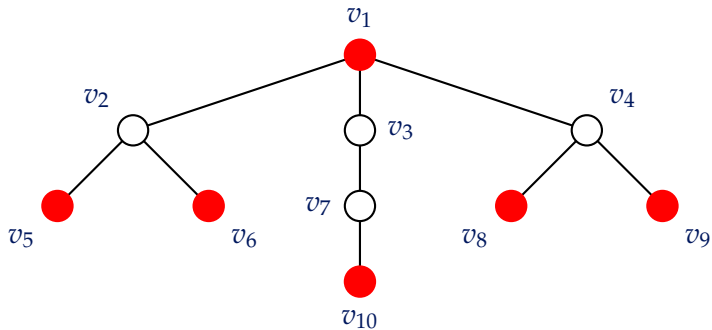
Nezávislá množina

Příklad: Strom



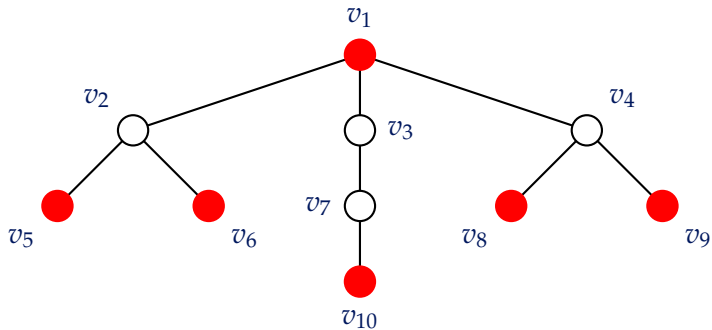
Nezávislá množina

Příklad: Strom



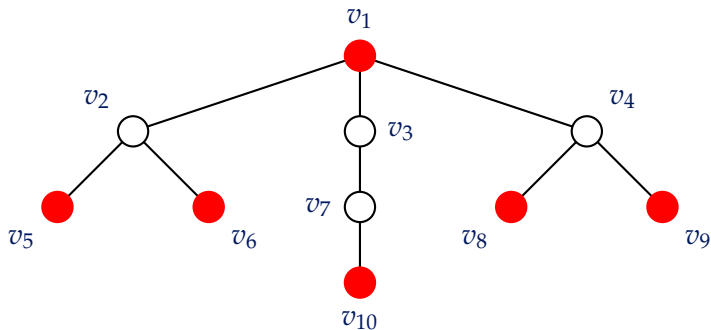
Nezávislá množina

Příklad: Strom



Nezávislá množina

Příklad: Strom



V obecném stromu T na n vrcholech platí pro největší nezávislá množina N následující odhad:

$$\left\lceil \frac{n}{2} \right\rceil \leq |N| \leq n - 1.$$

Problém nezávislé množiny v grafu

Problém nezávislé množiny v grafu

PROBLÉM NEZÁVISLÉ MNOŽINY V GRAFU (NzMna)

Vstup: Graf G a číslo $k \in \mathbb{N}$.

Otázka: Existuje v grafu G nezávislá množina velikosti alespoň k ?

Problém nezávislé množiny v grafu

PROBLÉM NEZÁVISLÉ MNOŽINY V GRAFU (NzMna)

Vstup: Graf G a číslo $k \in \mathbb{N}$.

Otázka: Existuje v grafu G nezávislá množina velikosti alespoň k ?

Ukážeme, že problém NzMna je **stejně těžký** jako SAT.

Problém kliky v grafu

- PAPADIMITRIOU, Christos H. *Computational complexity*. Reading: Addison-Wesley, 1994. ISBN 0-201-53082-1.
- MAREŠ, Martin a VALLA, Tomáš. *Průvodce labyrintem algoritmů*. Druhé vydání. CZ.NIC. Praha: CZ.NIC, z.s.p.o., 2022. ISBN 978-80-88168-63-8.